

**LILA – LLM-based Intelligent LLVM Assistant
per la generazione e debugging di programmi in Grammo**

**UNIVERSITÀ DEGLI STUDI DI SALERNO
CORSO: INGEGNERIA DEI LINGUAGGI
DI PROGRAMMAZIONE
2025/2026**

Prof. Gennaro Costagliola

Mario Cosenza
NF22500094

INDICE

1. Introduzione e Contesto Progettuale.....	2
2. Specifiche del Linguaggio Grammo.....	3
2.2 Semantica e Vincoli Logici.....	4
3. Architettura del Sistema Multi-Agente.....	5
3.1 Il Grafo di Stato (State Graph).....	5
3.2 Dettaglio degli Agenti.....	6
A. L'Orchestrator: Il Router Semantico.....	6
B. Il Planner: L'Architetto Strategico (Human-in-the-Loop).....	7
C. L'Integrator: Il Gestore del Workspace.....	7
D. Il Generator: Lo Specialista Sintattico.....	8
E. Il Tester: Il Validatore Avversario.....	8
F. Il Debugger Evaluator: Il Chirurgo del Codice.....	9
3.3 Disaccoppiamento del Testing: Il Microservizio FastAPI.....	9
4. Tools Deterministici e Controllo Qualità.....	12
4.1 Analisi Statica: Il Parser Lark.....	12
4.2 Compilazione JIT con LLVM Lite.....	12
4.3 Esempi di codice in Grammo.....	13
4.4 Testing Automatizzato.....	13
5. Piano di Esecuzione: Strategie AI e Ottimizzazione.....	13
5.1 Selezione dei Modelli: Approccio Ibrido.....	13
6. Interfaccia Utente: La Command Line Interface (CLI).....	14
6.1 UX e Flusso di Interazione.....	14
7. Deep Dive Tecnologico: Lo Stack di Orchestrazione.....	15
7.1 LangChain: Il Middleware Cognitivo.....	15
7.2 LangGraph: Oltre le Catene Lineari.....	15
8. Strategie di Inferenza: Modelli Locali e Open Source.....	16
8.1 La scelta di "gpt-oss-20b" e Gemma 3.....	16
8.2 Configurazione e Ottimizzazione Locale.....	17
8.3 Limiti dell'Approccio Locale.....	17
9. Guida alla Configurazione e Utilizzo.....	18
9.1 Preparazione dell'Ambiente.....	18
9.2 Avvio dei Servizi.....	18
10. Conclusioni, Limitazioni e Sviluppi Futuri.....	19
10.1 Analisi delle Limitazioni Tecniche.....	19
10.2 Risultati Comparativi e Metriche.....	20

1. Introduzione e Contesto Progettuale

Il panorama dello sviluppo software moderno è stato radicalmente trasformato dall'avvento dei **Large Language Models** (LLM). Strumenti come **GitHub Copilot** o **ChatGPT** hanno dimostrato come l'intelligenza artificiale generativa possa assistere i programmatori.

Tuttavia, la maggior parte di questi strumenti opera come "scatole nere" su linguaggi mainstream, spesso mancando di precisione quando si tratta di linguaggi di dominio specifico (DSL) o linguaggi didattici con regole rigide e non convenzionali.

Il progetto **LILA (LLM Integrated Language Agent)** nasce per colmare questa lacuna nel contesto accademico.

L'obiettivo non è semplicemente generare codice, ma creare un'architettura software robusta che integri la natura probabilistica degli LLM con la natura deterministica dei compilatori. Questo progetto è stato sviluppato in stretta sinergia con il collega **Salvatore Di Martino**, autore del linguaggio **Grammo**.

Mentre il progetto Grammo si focalizza sulla costruzione del compilatore (dal parsing alla generazione di codice macchina tramite LLVM), LILA costruisce un livello di orchestrazione intelligente sopra di esso. L'approccio scelto è quello dei **Sistemi Multi-Agente (MAS)**.

Invece di affidarsi a un singolo prompt monolitico ("Scrivi questo programma"), LILA scompone il **processo di sviluppo in fasi distinte**: pianificazione, scrittura, compilazione, test e debug, assegnando ogni fase a un **agente specializzato**. Questa separazione delle responsabilità permette un controllo più fine sulla qualità del codice e, soprattutto, abilita meccanismi di **auto-correzione** (self-healing) che sono il vero cuore dell'innovazione proposta.

2. Specifiche del Linguaggio Grammo

Per comprendere le sfide affrontate dall'agente intelligente, è necessario analizzare in dettaglio il linguaggio target.

Grammo non è un semplice dialetto del C, ma un **linguaggio imperativo** tipizzato staticamente con regole sintattiche stringenti.

2.1 Analisi Lessicale e Sintattica

Il linguaggio Grammo impone una struttura rigida che l'LLM deve "imparare" attraverso il **System Prompt** (definiti in [agents/prompts](#)).

Gestione dei Tipi e delle Variabili

A differenza di linguaggi a tipizzazione dinamica come Python, Grammo richiede una dichiarazione esplicita dei tipi. Il sistema di tipi include primitive essenziali quali **int** (interi), **real** (virgola mobile), **bool** (booleani) e **string**.

Una peculiarità sintattica che l'agente deve rispettare è la sintassi di dichiarazione: la keyword **var** è obbligatoria, seguita dal tipo, separato dai nomi delle variabili da due punti. Ad esempio, una dichiarazione come **int x;** (valida in C) genererebbe un errore di parsing in Grammo.

La forma corretta è **var int: x;**. L'inizializzazione contestuale è permessa e supporta l'inferenza del tipo, una caratteristica che l'agente *Generator* sfrutta per rendere il codice più conciso.

Il Paradigma di Input/Output

La più grande "trappola" per un LLM addestrato su grandi corpus di codice C/C++ è l'I/O. Grammo rifiuta funzioni standard come **printf** o **scanf**. Introduce invece operatori dedicati che fungono da primitive del linguaggio:

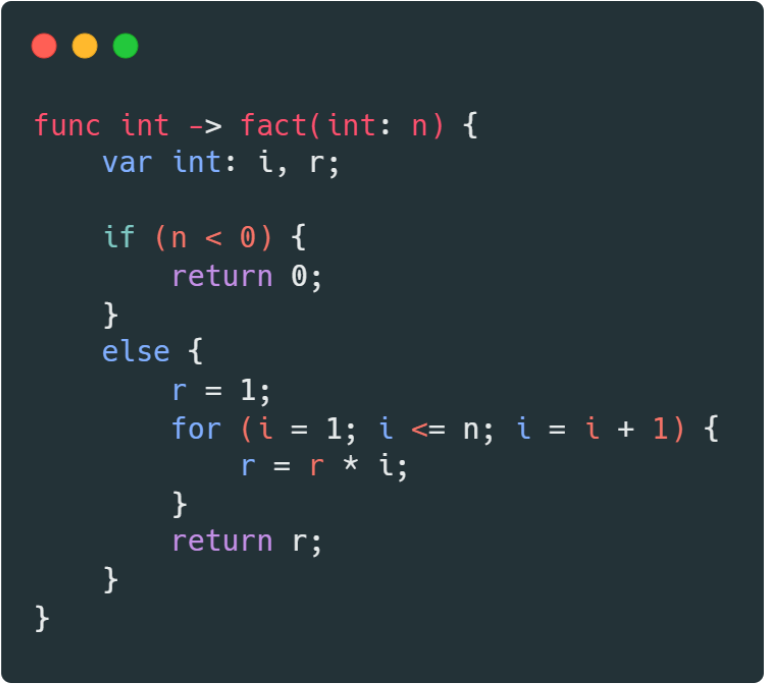
- **Operatore di Input (>>):** Questo operatore non è un bitwise shift, ma un comando di lettura. Richiede sintatticamente una stringa di prompt, un separatore obbligatorio **#** e la variabile di destinazione racchiusa tra parentesi. L'assenza delle parentesi o del cancelletto porta a un fallimento immediato del parser Lark.
- **Operatore di Output (<< e <<!):** Simmetricamente, l'output distingue tra stampa con e senza "a capo". Anche qui, la sintassi **<< "Messaggio" # (variabile);** è mandatoria.

2.2 Semantica e Vincoli Logici

L'architettura di LILA deve tenere conto non solo della forma, ma anche del significato del codice. Il compilatore Grammo (integrato nel sistema) applica uno **scoping lessicale statico**.

Una regola semantica critica implementata è il **divieto di shadowing**. Se l'agente LILA tenta di dichiarare una variabile locale con lo stesso nome di una variabile globale o di un argomento di funzione, il compilatore solleva un **SemanticError**. Inoltre, per le funzioni che restituiscono valori, è implementata una **Return Analysis**: il compilatore verifica che tutti i possibili rami di esecuzione (inclusi **if/else** annidati) terminino con un'istruzione **return**.

L'agente *Debugger* è stato specificamente istruito per riconoscere l'errore "Missing return statement" e correggere il flusso di controllo aggiungendo **return** di default o ristrutturando la logica condizionale.



```
func int -> fact(int: n) {  
    var int: i, r;  
  
    if (n < 0) {  
        return 0;  
    }  
    else {  
        r = 1;  
        for (i = 1; i <= n; i = i + 1) {  
            r = r * i;  
        }  
        return r;  
    }  
}
```

3. Architettura del Sistema Multi-Agente

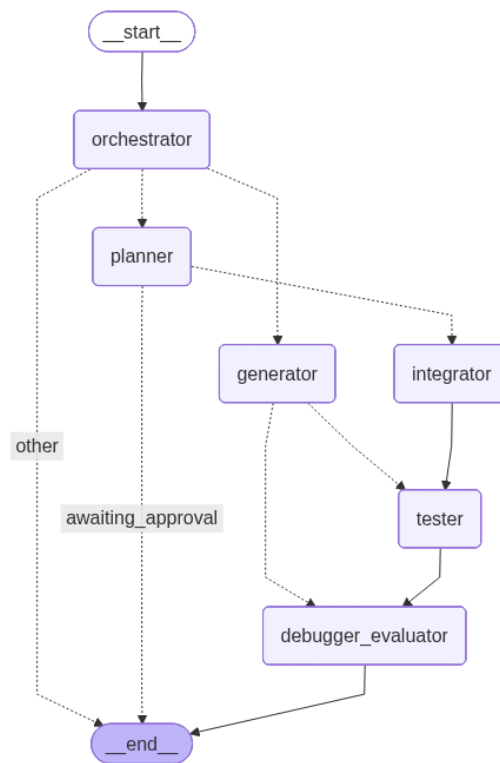
L'architettura di LILA non è un semplice script lineare, ma un grafo di stati complesso gestito dalla libreria **LangGraph**. Questa scelta progettuale permette di definire cicli, condizioni di uscita e persistenza della memoria.

3.1 Il Grafo di Stato (State Graph)

Il cuore del sistema è definito nella funzione `build_llm` e nella classe `_GeminiSafeWrapper` (file `agents/multi_agent.py`). La struttura dati che fluisce attraverso il grafo è l'`AgentState`, un dizionario tipizzato (`TypedDict`) che funge da memoria condivisa. Questo stato contiene:

1. `messages`: La cronologia completa della conversazione (fondamentale per mantenere il contesto tra i turni).
2. `code`: L'ultima versione del codice sorgente generato.
3. `errors`: Una lista strutturata degli errori riscontrati (siano essi di parsing, compilazione o test).
4. `plan`: Il piano di esecuzione generato dall'agente Planner.

Il flusso di esecuzione è dinamico. Il nodo iniziale, l'**Orchestrator**, analizza l'intento dell'utente. Se la richiesta è banale ("Correggi questo errore di sintassi"), instrada direttamente al *Generator*. Se la richiesta è complessa ("Crea un sistema di gestione studenti"), attiva il *Planner*.



3.2 Dettaglio degli Agenti

L'architettura di LILA non si basa su un unico modello "tuttofare", ma su una costellazione di agenti specializzati. Questa scelta progettuale, nota in letteratura come **"Society of Agents"**, permette di aggirare i **limiti di attenzione** degli LLM: invece di caricare un unico contesto con istruzioni contrastanti (pianificazione, sintassi, testing), ogni agente riceve un "System Prompt" verticale che definisce rigidamente il suo ruolo, i suoi vincoli e il suo formato di output. Di seguito viene analizzato di ciascun agente:

A. L'Orchestrator: Il Router Semantico

L'**Orchestrator** agisce come la corteccia prefrontale del sistema. Il suo compito non è risolvere il problema, ma **comprenderlo**.

Riceve l'input in linguaggio naturale dell'utente e deve **classificarlo in una tassonomia** predefinita per attivare il ramo corretto del grafo.

Il suo prompt di sistema impone di non generare mai codice, ma di agire come un dispatcher. Un estratto delle sue istruzioni recita:

System Instruction Extract:

*"Sei un classificatore di intenti per il sistema di sviluppo LILA. Analizza la richiesta dell'utente e decidi il percorso di esecuzione: 1. **PLANNER**: Se la*

*richiesta è complessa, vaga o richiede più funzioni (es. 'Crea un gioco'). 2. **GENERATOR**: Se la richiesta è atomica o specifica su un singolo file (es. 'Scrivi una funzione somma'). 3. **QUESTION**: Se l'utente chiede spiegazioni teoriche. Non rispondere alla domanda, restituisci solo la label di routing."*

Questa separazione previene che il sistema inizi a scrivere codice prematuramente su richieste malformate.

B. Il Planner: L'Architetto Strategico (Human-in-the-Loop)

Il **Planner** è l'agente più astratto. Non conosce la sintassi esatta di **Grammo** (o meglio, non è tenuto a rispettarla rigorosamente), ma possiede forti capacità di ragionamento logico.

La sua funzione critica è la **decomposizione del problema**. Per richieste come "Crea una calcolatrice", il Planner non si getta nella scrittura, ma produce un **documento strutturato (JSON)** che elenca i passaggi.

Il prompt del Planner forza un output strutturato per garantire che il piano sia leggibile dalla macchina:

System Instruction Extract:

*"Non scrivere codice. Sei un Software Architect. Decomponi il problema dell'utente in sotto-task logici. Il tuo output deve essere tassativamente un oggetto JSON contenente una lista **steps**. Ogni step deve definire: - **function_name**: Il nome della funzione da implementare. - **description**: Cosa fa la funzione. - **dependencies**: Quali variabili globali tocca. Attendi l'approvazione dell'utente prima di procedere."*

L'aspetto innovativo qui è il meccanismo di **"Stop-and-Wait"**: il grafo si congela dopo che il Planner ha parlato. L'utente deve rivedere il piano.

Questo feedback loop umano risparmia risorse computazionali enormi, evitando di generare intere codebase basate su un fraintendimento iniziale.

C. L'Integrator: Il Gestore del Workspace

Mentre il **Generator** lavora su singoli file, l'**Integrator** ha una visione olistica. È l'agente introdotto per gestire la complessità multi-file e la coerenza dello stato globale. Il suo ruolo è prendere i **pezzi prodotti o pianificati e assemblarli in un unico "Workspace" coerente**. Il suo prompt è focalizzato sulla visibilità delle variabili e sulla coerenza delle firme delle funzioni.

System Instruction Extract:

"Il tuo compito è assemblare il codice. Assicurati che le variabili dichiarate come `var int: global_counter` siano visibili alle funzioni che le utilizzano. Verifica che non ci siano definizioni duplicate tra i vari blocchi. Tu sei il garante della coerenza semantica del progetto intero."

D. Il Generator: Lo Specialista Sintattico

Il **Generator** riceve un singolo task dal piano (es. "Implementa la funzione fattoriale") e deve tradurlo in codice **Grammo** compilabile. Il suo System Prompt è il più voluminoso del sistema, poiché funge da manuale di riferimento dinamico. Contiene l'intera specifica EBNF del linguaggio e, soprattutto, esempi "Few-Shot" di cosa fare e cosa *non* fare.

Per combattere le allucinazioni derivanti dal training su C/C++, il prompt enfatizza le differenze critiche:

System Instruction Extract (Focus Sintassi):

"Sei un esperto sviluppatore Grammo. Dimentica la sintassi C standard. REGOLE IMPERATIVE: 1. Input: Usa SOLO `>> \"Prompt\" # (var);`. Non usare `scanf`. 2. Output: Usa SOLO `<< \"Label\" # (var);`. Non usare `printf`. 3. Variabili: Dichiara sempre con `var tipo: nome;`. Prima di restituire il codice, invoca il tool `grammo_lark` per validare la tua stessa sintassi."

L'istruzione finale è cruciale: istruisce l'agente a usare i tool deterministici per auto-correggersi *prima* ancora di mostrare il risultato all'utente.

E. Il Tester: Il Validatore Avversario

Il **Tester** ha una **personalità "avversaria"**. Il suo obiettivo è rompere il codice generato. Non si limita a verificare se il codice compila, ma **progetta casi di test** (Input/Output Pairs) per verificare la robustezza logica.

Il prompt del Tester richiede di generare scenari limite (edge cases).

System Instruction Extract:

"Analizza il codice fornito. Genera una suite di test JSON. Includi casi standard (es. input positivi) e casi limite (es. zero, numeri negativi). Il tuo output sarà

inviato a un runner di test isolato. Non eseguire tu il codice, definisci solo i vincoli di accettazione."

F. Il Debugger Evaluator: Il Chirurgo del Codice

Il componente più sofisticato è il **Debugger**. Viene attivato solo in caso di fallimento (errore di compilazione o test fallito). A differenza di un generico "Fixer", il Debugger riceve in input una tripla di dati: {Codice Originale, Errore del Compilatore, Linea dell'Errore}.

Il suo prompt è progettato per minimizzare le modifiche distruttive:

System Instruction Extract:

"Hai ricevuto un errore di compilazione: {error_message} alla riga {line}. Analizza SOLO la sezione colpevole. Non riscrivere l'intero programma se non necessario. Applica una patch chirurgica. Spiega il tuo ragionamento: 'Ho visto che mancava il cancelletto nell'istruzione di stampa, quindi ho corretto in...' "

Questa specializzazione del prompt trasforma il Debugger da un semplice rigeneratore di testo a un vero strumento di analisi statica assistita, capace di apprendere dagli errori specifici sollevati dal backend LLVM.

3.3 Disaccoppiamento del Testing: Il Microservizio FastAPI

Mentre la CLI e gli agenti gestiscono la logica, l'esecuzione del codice potenzialmente instabile (il codice generato dall'AI) è isolata in un **microservizio** separato.

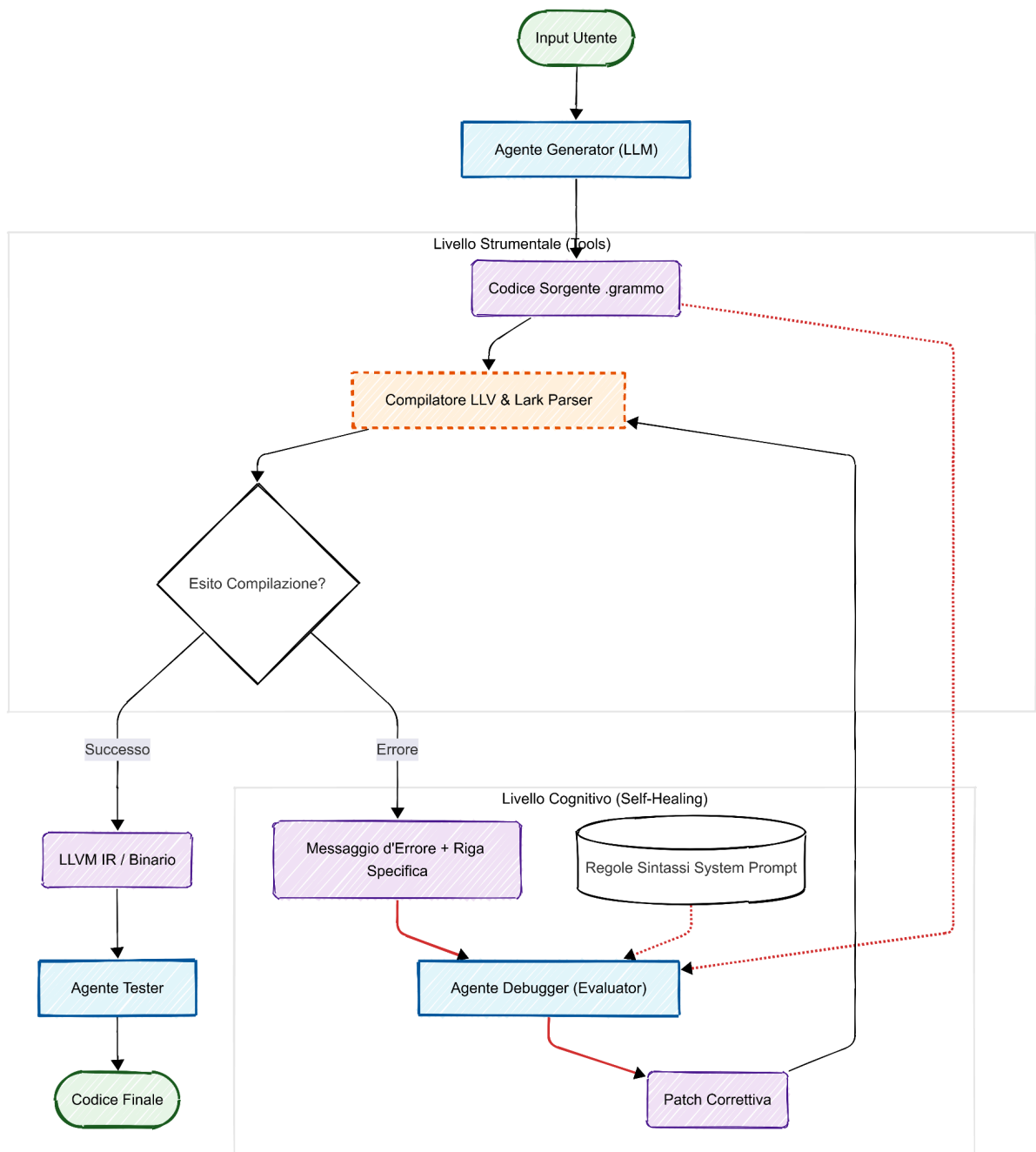
Analizzando `orchestrator.py`, notiamo la configurazione `TESTER_URL = "http://127.0.0.1:8088/invoke"`. Questo rivela che il nodo `Tester` (`agents/tester.py`) non è altro che un client HTTP. Il vero motore di testing è un server **FastAPI** dedicato.

Perché questa scelta architetturale?

1. **Sandbox di Sicurezza:** Eseguire codice generato da un LLM (che viene compilato JIT ed eseguito in memoria) comporta rischi. Separando il Tester in un processo server distinto, un eventuale crash (segfault) del codice Grammo farà cadere solo il worker FastAPI, non l'intero orchestratore LILA, preservando lo stato della conversazione.

2. **Scalabilità:** Il server di testing può essere deployato su una macchina diversa (o un container Docker) rispetto all'agente, permettendo di parallelizzare i test se necessario.
3. **Indipendenza Tecnologica:** Il server FastAPI espone endpoint standard. Questo significa che l'agente può inviare il codice e i casi di test e ricevere indietro un JSON strutturato (**pass**, **fail**, **stdout**), senza dover gestire le complessità dei subprocess Python o della cattura dei buffer di output all'interno del loop di LangGraph.

Schema generazione codice con LILA



4. Tools Deterministici e Controllo Qualità

LILA non si limita a "immaginare" che il codice funzioni; lo prova scientificamente. L'integrazione avviene tramite il protocollo **MCP (Model Context Protocol)** e server locali. Senza MCP, ogni integrazione richiederebbe di scrivere codice "colla" personalizzato all'interno del prompt dell'agente. Questo rende il sistema fragile e difficile da mantenere.

LILA implementa un server MCP ([mcp/server.py](#)) che espone le funzionalità del compilatore come risorse standardizzate.

- **Disaccoppiamento:** L'agente non sa *come* funziona il compilatore Grammo in Python. Sa solo che esiste un tool chiamato [grammo_compile](#) che accetta una stringa e restituisce un JSON.
- **Sicurezza:** Il server MCP agisce da sandbox. L'agente non può eseguire comandi arbitrari sul sistema operativo (come `rm -rf`), ma può solo invocare le funzioni esplicitamente esposte dal protocollo.
- **Estensibilità:** Se in futuro il collega di Grammo aggiornasse il compilatore, basterebbe aggiornare il server MCP; l'agente LILA continuerebbe a funzionare senza modifiche al codice sorgente dell'IA.

4.1 Analisi Statica: Il Parser Lark

Come da specifiche di progetto, l'analizzatore sintattico prodotto tramite **Lark** è stato integrato obbligatoriamente. Nel file [mcp/server.py](#), è esposta la funzione [validate_syntax](#). Quando l'agente invoca questo tool, il codice Grammo viene parsato per costruire l'albero sintattico (AST).

Se Lark solleva un'eccezione ([UnexpectedToken](#), [UnexpectedCharacters](#)), questa viene catturata, formattata in modo leggibile e restituita all'agente. Questo filtro preventivo è essenziale: impedisce di inviare al compilatore LLVM codice che è strutturalmente invalido, ottimizzando i tempi di risposta.

4.2 Compilazione JIT con LLVM Lite

Il "motore" del sistema è l'interfaccia con LLVM. Utilizzando la libreria [llvmlite](#) (binding Python per LLVM), LILA è in grado di compilare il codice Grammo in rappresentazione intermedia (IR) e successivamente in codice macchina, eseguendolo in memoria (Just-In-Time).

Il tool [grammo_compile](#) esposto agli agenti non si limita a dire "Compilato". Restituisce un oggetto complesso contenente:

- **Il codice IR generato** (utile per il debugging avanzato).
- **Eventuali warning semantici.**
- **L'esito dell'esecuzione se richiesta.**

4.3 Esempi di codice in Grammo

Grazie ai tool [grammo_list_examples](#) e [grammo_read_example](#), il generatore può consultare esempi di codice come 'Calcolatrice' o 'Fibonacci', che **illustrano** i principali costrutti di Grammo. Ciò aiuta a ridurre le allucinazioni, favorendo il *grounding* del modello su una **fonte di verità** affidabile.

4.4 Testing Automatizzato

La qualità del codice è garantita dal **Tester Server** ([tester_server.py](#)). Questo componente agisce come un'entità indipendente. Quando viene generato un programma (es. "Calcolatrice"), il Tester:

1. **Analizza la specifica del problema.**
2. **Genera** (tramite un LLM locale separato, es. Gemma 3) una serie di casi di test (Input: "2+2", Output Atteso: "4").
3. **Esegue** il binario compilato JIT iniettando gli input e catturando lo standard output.
4. **Confronta l'output reale con quello atteso.** Solo se tutti i test passano, il codice viene marcato come "Final".

5. Piano di Esecuzione: Strategie AI e Ottimizzazione

La realizzazione di **LILA** ha richiesto decisioni architetturali precise riguardo l'uso delle risorse computazionali e dei modelli linguistici.

5.1 Selezione dei Modelli: Approccio Ibrido

Per bilanciare costi, privacy e prestazioni, è stata adottata una strategia ibrida documentata nel file [USER_GUIDE.md](#).

- **Il "Cervello" nel Cloud (Gemma 3 27B/Gemini 3 Pro/Flash):** Per i nodi *Orchestrator* e *Debugger* è stato scelto Google Gemini (via API). La motivazione risiede nella sua finestra di contesto superiore e nelle capacità di ragionamento logico. Il debugging richiede di tenere in memoria l'intero file sorgente, l'errore e la documentazione del linguaggio contemporaneamente; modelli più piccoli tendono a "dimenticare" istruzioni in questo scenario saturo.
- **La "Forza Lavoro" Locale (GPT-OSS-20B):** Per il *Tester* e task ripetitivi, il sistema supporta l'esecuzione locale tramite **Ollama**. Eseguire i test in locale garantisce che i dati sensibili dei test non lascino la macchina e riduce la latenza per operazioni ad alto volume.

6. Interfaccia Utente: La Command Line Interface (CLI)

Sebbene LILA sia un sistema complesso di agenti intelligenti, l'interfaccia utente è stata deliberatamente mantenuta essenziale e orientata allo sviluppatore: una **Command Line Interface (CLI)** interattiva.

La scelta di non sviluppare una GUI web (come React o Streamlit) ma di rimanere nel terminale risponde a due principi cardine del progetto:

1. **Isomorfismo con il Compilatore:** Poiché il compilatore Grammo e i tool di build (LLVM) operano da riga di comando, l'agente deve "vivere" nello stesso ambiente in cui il codice viene compilato ed eseguito.
2. **Leggerezza e Portabilità:** Il client (`agents/agent_client.py`) è un processo leggero che comunica con il backend tramite socket/HTTP, permettendo di eseguire l'interfaccia anche su macchine con risorse limitate, delegando l'inferenza pesante al server.

6.1 UX e Flusso di Interazione

L'interazione avviene tramite un loop **REPL** (Read-Eval-Print Loop) potenziato. All'avvio, l'utente viene accolto dal banner di LILA e da un prompt di input.

```
Grammo Console Client
LangGraph Streaming Interface

Commands: ':quit' to exit, ':new' new session, ':trace off|basic|debug'

User > 
```

Quando l'utente inserisce un task (es. *"Crea un programma che calcola l'area del cerchio di raggio 2"*), la CLI non si blocca in attesa della risposta finale. Al contrario, essa sottoscrive lo **stream di eventi** proveniente da LangGraph.

```
function.\nINFO: Compilation successful.\n\n== LLVM IR (Optimized) ==\nModuleID = '<string>...
* compile_errors: []
* validated_code: SUMMARY: This program calculates the area of a circle with a radius of 2.\nTESTS: Passed 1/1\nERRORS: []\nfunc real ->
calculate_circle_area() {\n  var real: radius, area;\n  radius = 2.0;\n  area = 3.14159 * ra...
* validation_summary: This program calculates the area of a circle with a radius of 2.
* max_iters: 8
End Trace

Assistant
SUMMARY: This program calculates the area of a circle with a radius of 2.

func real -> calculate_circle_area() {
  var real: radius, area;
  radius = 2.0;
  area = 3.14159 * radius * radius;
  <<! "Area of the circle: " # (area);
  return area;
}
```

7. Deep Dive Tecnologico: Lo Stack di Orchestrazione

L'intelligenza di LILA non risiede nel singolo modello linguistico, ma nell'architettura che lo governa. Di seguito viene analizzato in dettaglio lo stack tecnologico, giustificando le scelte rispetto alle alternative di mercato.

7.1 LangChain: Il Middleware Cognitivo

Alla base del sistema c'è **LangChain**, utilizzato non come semplice wrapper per le API, ma come framework per la gestione dei prompt e dei tool. In LILA, LangChain gestisce l'astrazione dei modelli (**ChatGoogleGenerativeAI** per Gemini, **ChatOllama** per i modelli locali). Questo ci permette di cambiare il "cervello" del sistema modificando una sola riga nel file `.env`, senza riscrivere la logica applicativa. Inoltre, le **PromptTemplates** di LangChain permettono di iniettare dinamicamente le regole sintattiche di Grammo nel contesto, garantendo che l'LLM abbia sempre "sott'occhio" la documentazione aggiornata.

7.2 LangGraph: Oltre le Catene Lineari

La vera spina dorsale di LILA è **LangGraph**. Nelle prime fasi dell'IA generativa, si usavano le "Chains" (catene sequenziali): *Input* → *LLM* → *Output*.

Questo approccio è fallimentare per la generazione di codice, che è intrinsecamente un processo ciclico e iterativo (*Scrivi* → *Compila* → *Errore* → *Correggi* → *Ricompila*).

LangGraph introduce il concetto di **Grafo di Stato Ciclico**.

- **Perché LangGraph?** A differenza di una macchina a stati finiti (FSM) tradizionale hard-coded in Python, LangGraph persiste lo stato (**AgentState**) tra un passo e l'altro. Questo significa che l'agente può "ricordare" i tentativi falliti precedenti. Se il Debugger prova una soluzione e fallisce, il grafo mantiene la storia di quell'errore per evitare di ripeterlo nel ciclo successivo.
- **Implementazione in LILA:** Nel file `agents/multi_agent.py`, il grafo è definito con nodi condizionali. L'arco che esce dal nodo **Compiler** non è fisso: è un arco dinamico che, basandosi sulla variabile `compile_result['success']`, decide se avanzare al **Tester** o retrocedere al **Debugger**. Questa logica condizionale è ciò che conferisce a LILA un comportamento "agente" e non puramente "generativo".

8. Strategie di Inferenza: Modelli Locali e Open Source

Un requisito chiave di LILA l'indipendenza da **API costose**. LILA supporta l'esecuzione su modelli Open Weights (comunemente detti Open Source) tramite **Ollama**.

8.1 La scelta di "gpt-oss-20b" e Gemma 3

Durante lo sviluppo sono stati testati diversi modelli della classe **20B-30B parametri**. Il modello di riferimento citato, una variante della famiglia **gpt-oss** o **Gemma 3 27b**, rappresenta il "sweet spot" attuale per l'hardware consumer di come la **RTX 3070** usata nel laboratorio di test.

Perché 20-27 Miliardi di Parametri?

1. Ragionamento Complesso:

I modelli da 7B o 8B (come Llama 3 8B) sono eccellenti nel chat, ma faticano a mantenere la coerenza logica necessaria per rispettare la sintassi rigida di Grammo (es. non dimenticare il cancelletto **#** nell'operatore **>>**). I modelli da 20B+ possiedono una capacità di attenzione sufficiente per gestire le dipendenze a lungo raggio nel codice.

2. Gestione delle Istruzioni (Instruction Following):

Il modello deve seguire il System Prompt che descrive la grammatica EBNF. I modelli più piccoli tendono a "dimenticare" le istruzioni di sistema quando la conversazione si allunga (fenomeno del *catastrophic forgetting* nel contesto).

3. Fattibilità Hardware:

Un modello da 27B quantizzato a 4-bit (Q4_K_M) richiede circa 8-16 GB di VRAM. Questo lo rende eseguibile su workstation comuni senza ricorrere a cluster H100 costosissimi.

8.2 Configurazione e Ottimizzazione Locale

L'integrazione locale avviene tramite **Ollama**, che funge da backend di inferenza. La configurazione è gestita nel file `.env`:

Snippet di codice

```
None
LLM_BACKEND=ollama

OLLAMA_MODEL=gemma3:27b-instruct-q4_0

OLLAMA_BASE_URL=http://localhost:11434
```

Tecnica di "System Patching":

Una sfida tecnica significativa riscontrata con i modelli open source (incluso [gpt-oss-20b](#) e [Gemma](#)) è la gestione incoerente dei ruoli "System". Molti modelli open source sono ottimizzati per chat User/Assistant e ignorano i messaggi System profondi.

Per ovviare a ciò, LILA implementa in [agents/multi_agent.py](#) una funzione middleware `_merge_system_for_local`.

Questa funzione intercetta la chiamata prima che parta verso Ollama, estrae il System Prompt (contenente le regole di Grammo) e lo fonde ("prepend") al primo messaggio dell'utente. Questo "trucco" di prompt engineering ha aumentato il tasso di rispetto della sintassi nei test locali.

8.3 Limiti dell'Approccio Locale

Nonostante l'ottimizzazione, l'uso di modelli locali 20B presenta limitazioni rispetto a GPT-5.2 o Gemini 3 Pro:

- Finestra di Contesto:** Mentre Gemini supporta 1M+ token, i modelli locali spesso degradano le prestazioni oltre i 4k-8k token. Per programmi Grammo molto lunghi, il modello potrebbe perdere la definizione delle variabili dichiarate all'inizio del file.
- Velocità di Inferenza:** Su una GPU singola, la generazione di codice complesso può richiedere fino a 10 minuti, contro 30-60 secondi delle API Cloud.
- Allucinazioni Sintattiche:** I modelli open source hanno visto milioni di righe di C e C++. Hanno un forte bias verso la sintassi C standard. Spesso "allucinano" mettendo un `printf` invece di `<<!`, richiedendo l'intervento del Debugger per correggere l'errore.

9. Guida alla Configurazione e Utilizzo

Per replicare l'ambiente di esecuzione descritto, seguire la procedura rigorosa definita in [USER_GUIDE.md](#).

9.1 Preparazione dell'Ambiente

Il progetto utilizza [conda](#) per garantire l'isolamento delle dipendenze. Il file [environment.yml](#) blocca le versioni critiche:

YAML

```
None
dependencies:

- python=3.11

- pip

- pip:

- langgraph>=0.0.10

- langchain-ollama

- llvmlite
```

L'uso di Python 3.11 è mandatorio per la compatibilità con le ultime feature asincrone di LangGraph.

9.2 Avvio dei Servizi

L'architettura a microservizi richiede un avvio coordinato. È stato creato uno script PowerShell [start_services.ps1](#) che automatizza il bootstrap:

1. **Avvia il Server MCP** (per il compilatore).
2. **Avvia il Server di Testing** (su porta distinta).
3. Attende l'**health-check** positivo dei servizi.
4. **Lancia il Client CLI**.

PowerShell

None

```
.\start_services.ps1 -Local -Model "gemma3:27b"
```

Questo comando permette di testare il sistema in modalità completamente offline, facendo affidamento ai modelli installati con Ollama.

10. Conclusioni, Limitazioni e Sviluppi Futuri

Il progetto **LILA** rappresenta un tentativo concreto di ingegnerizzare l'interazione con i Large Language Models, spostando il focus dal semplice "prompt engineering" alla costruzione di **Compound AI Systems**.

Lavorare in sinergia con il progetto **Grammo** ha permesso di evidenziare una verità fondamentale dello sviluppo assistito da AI: la potenza del modello linguistico è nulla senza il controllo di strumenti deterministici.

Attraverso l'implementazione di un grafo di stati con **LangGraph** e l'integrazione del compilatore tramite **MCP**, LILA ha dimostrato che è possibile generare codice corretto per un linguaggio *custom*, sconosciuto al training set del modello, dotandolo degli strumenti per auto-correggersi.

10.1 Analisi delle Limitazioni Tecniche

Nonostante il successo nel raggiungere gli obiettivi funzionali (generazione, compilazione e test), il sistema presenta limitazioni intrinseche che sono emerse chiaramente durante la fase di valutazione sperimentale.

1. Il Bias di Addestramento e le "Allucinazioni Sintattiche"

La sfida più ardua affrontata da LILA non è stata la logica algoritmica, ma la sintassi specifica di I/O di Grammo. I modelli utilizzati (sia Gemini che Gemma) sono stati addestrati su terabyte di codice C, C++ e Java. Di conseguenza, possiedono un **forte bias** statistico verso l'uso di **printf**, **cout** o **scanf**. Nonostante il System Prompt contenga la specifica EBNF corretta (**>>** e **<<**), il modello tende a "ricadere" nelle sue abitudini apprese durante il pre-training quando la finestra di contesto si satura o il task diventa complesso.

Nei log di debug, si osserva spesso che il **Generator** scrive correttamente la logica ma **fallisce l'I/O**. È solo grazie all'intervento del **Debugger** (che legge l'errore di Lark "Unexpected Token") che il codice viene corretto.

Questo conferma che un approccio "Zero-Shot" è insufficiente per DSL (Domain Specific Languages) con sintassi non convenzionale.

2. Latenza del Ciclo Agentico

L'architettura "Human-in-the-loop" e il ciclo di auto-correzione introducono una latenza significativa. Mentre un completamento di codice standard (stile Copilot) richiede alcuni secondi, un task completo su LILA (Pianificazione → Generazione → Compilazione → Fix → Test) può richiedere dai 3 ai 10 minuti. Questa latenza è dovuta principalmente ai tempi di *round-trip* delle chiamate al compilatore e alla necessità di ri-generare l'intero file sorgente ad ogni correzione. In uno scenario di produzione, questo tempo di attesa sarebbe accettabile solo per task complessi..

3. Dipendenza dal Prompt di Sistema nei Modelli Locali

Come discusso nella sezione tecnica, i modelli Open Weights di classe 20B-30B (come Gemma 3) mostrano una fragilità strutturale nel seguire istruzioni di sistema complesse ("**System Prompt Adherence**"). La necessità di implementare il "patching" del prompt nel file [multi_agent.py](#) dimostra che l'architettura dei modelli attuali è ancora fortemente ottimizzata per la chat conversazionale piuttosto che per l'esecuzione rigorosa di istruzioni tecniche nascoste. Questo limita la portabilità del sistema: cambiare il modello sottostante richiede spesso una **ricalibrazione manuale dei prompt**.

10.2 Risultati Comparativi e Metriche

L'efficacia di LILA è stata misurata qualitativamente osservando il tasso di successo (Pass Rate) su due categorie di problemi:

- **Algoritmi Standard (es. Fibonacci, Fattoriale):**
Il sistema raggiunge un tasso di successo prossimo al **90% al primo o secondo tentativo**. La logica è nota al modello, e l'agente deve solo "tradurre" la sintassi.
- **Logica Custom (es. Calcolatrice con I/O specifico):**
Qui il tasso di successo "Zero-Shot" (senza correzione) scende drasticamente sotto il **30%**.

10.3 Sviluppi Futuri

Per evolvere LILA da prototipo accademico a strumento di sviluppo professionale, si identificano tre direzioni di ricerca future:

1. Fine-Tuning (LoRA):

Invece di affidarsi interamente al Prompt Engineering (In-Context Learning), la soluzione definitiva al problema del bias sintattico sarebbe il *Fine-Tuning* del modello. Creare un dataset di 1000-2000 coppie (Istruzione, Codice Grammo) e addestrare un adattatore LoRA (Low-Rank Adaptation) su Gemma permetterebbe al modello di "interiorizzare" la sintassi `<< / >>`, riducendo drasticamente la necessità dell'intervento del Debugger e abbattendo la latenza.

2. Integrazione LSP (Language Server Protocol):

Attualmente LILA vive in una CLI. Il passo successivo sarebbe incapsulare il client in un'estensione per VS Code che agisca come un Language Server. Questo permetterebbe di suggerire fix direttamente nell'editor dell'utente, intercettando gli errori di compilazione in background e proponendo "Quick Fix" generati dall'agente senza interrompere il flusso di lavoro.

3. Memoria a Lungo Termine (RAG):

Per progetti di grandi dimensioni, l'agente dovrebbe avere accesso a una memoria vettoriale contenente non solo la grammatica, ma anche esempi di codice generati in precedenza e librerie standard del linguaggio Grammo, permettendo il riutilizzo di funzioni già validate.